Lab 6

Artificial Intelligence Lab

1 2 3 4

Program 1

Best First Search Traversal on a Graph

Perform Best First Search on the given weighted graph from Node S to Node G using the provided heuristic values h(n). Print the order of nodes explored and the path found.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_06 > Q1.py > best_first_search

```python
import heapq
# Graph with costs
graph = {
    'S': [('A', 3), ('B', 5)],
    'A': [('S', 3), ('C', 4), ('D', 2)],
    'B': [('S', 5), ('D', 6), ('G', 9)],
    'C': [('A', 4), ('G', 7)],
    'D': [('A', 2), ('B', 6), ('E', 1)],
    'E': [('D', 1), ('G', 3)],
    'G': []
}
# Heuristic values
h = {'S': 10, 'A': 7, 'B': 6, 'C': 5, 'D': 4, 'E': 2, 'G': 0}

def best_first_search(start, goal):
    visited = set()
    pq = [(h[start], start, [start], 0)]  # (heuristic, node, path, cost)

    while pq:
        _, node, path, cost = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)
        print(node, end=" ")

        if node == goal:
            print("\nPath:", " -> ".join(path))
            print("Total Cost:", cost)
            return

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq, (h[neighbor], neighbor, path + [neighbor], cost + weight))

best_first_search('S', 'G')
```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gaurav
.py"
S B G
Path: S -> B -> G
Total Cost: 14
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 2

Route Finding Between Two Cities

Given a road network with travel costs and straight-line distances to the goal, use Best First Search to find a route from City S to City G. Print the path and total cost incurred.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_06 > Q2.py > ...

```python
import heapq

graph = {
    'S': [('A', 3), ('B', 5)],
    'A': [('C', 4), ('D', 2)],
    'B': [('D', 6), ('G', 9)],
    'C': [('G', 7)],
    'D': [('E', 1)],
    'E': [('G', 3)],
    'G': []
}

h = {'S': 10, 'A': 7, 'B': 6, 'C': 5, 'D': 4, 'E': 2, 'G': 0}

def best_first_route(start, goal):
    pq = [(h[start], start, [start], 0)]
    visited = set()

    while pq:
        _, node, path, cost = heapq.heappop(pq)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            print("Route:", " -> ".join(path))
            print("Total Cost:", cost)
            print("Cities explored:", len(visited))
            return

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq, (h[neighbor], neighbor, path + [neighbor], cost + weight))

best_first_route('S', 'G')
```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % pyth
.py"
Route: S -> B -> G
Total Cost: 14
Cities explored: 3
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 3

Water Jug Problem Using Best First Search

Given two jugs of capacity 4L and 3L, measure exactly 2L of water using Best First Search. Define a suitable heuristic, explore states, and print the sequence of actions from initial to goal state.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_06 > Q3.py > best_first_jug

```python
import heapq
capacity1 = 4
capacity2 = 3
goal = 2
def heuristic(state):
    return abs(goal - state[0])  # based on 4L jug
def get_neighbors(state):
    x, y = state
    neighbors = []
    # Fill
    neighbors.append((capacity1, y))
    neighbors.append((x, capacity2))
    # Empty
    neighbors.append((0, y))
    neighbors.append((x, 0))
    # Pour
    transfer = min(x, capacity2 - y)
    neighbors.append((x - transfer, y + transfer))
    transfer = min(y, capacity1 - x)
    neighbors.append((x + transfer, y - transfer))
    return neighbors
def best_first_jug():
    start = (0, 0)
    pq = [(heuristic(start), start, [start])]
    visited = set()
    while pq:
        _, state, path = heapq.heappop(pq)

        if state in visited:
            continue

        visited.add(state)

        if state[0] == goal:
            print("Solution Path:")
            for p in path:
                print(p)
            return

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                heapq.heappush(pq, (heuristic(neighbor), neighbor, path + [neighbor]))

best_first_jug()
```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gaur
.py"
Solution Path:
(0, 0)
(0, 3)
(3, 0)
(4, 0)
(1, 3)
(1, 0)
(0, 1)
(4, 1)
(2, 3)
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Program 4

Missionary-Cannibal Problem Using Best First Search

Three missionaries and three cannibals must cross a river using a boat that holds at most 2. Cannibals must never outnumber missionaries on either bank. Solve using Best First Search with a suitable heuristic.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_06 > Q4.py > best_first_mc

```python
import heapq
def is_valid(m, c):
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)
def heuristic(state):
    m, c, _ = state
    return m + c  # people left
def get_neighbors(state):
    m, c, boat = state
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
    neighbors = []

    for dm, dc in moves:
        if boat == 'L':
            new_m, new_c = m - dm, c - dc
            new_boat = 'R'
        else:
            new_m, new_c = m + dm, c + dc
            new_boat = 'L'

        if 0 <= new_m <= 3 and 0 <= new_c <= 3 and is_valid(new_m, new_c):
            neighbors.append((new_m, new_c, new_boat))

    return neighbors
def best_first_mc():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')
    pq = [(heuristic(start), start, [start])]
    visited = set()

    while pq:
        _, state, path = heapq.heappop(pq)

        if state in visited:
            continue

        visited.add(state)

        if state == goal:
            print("Solution Path:")
            for p in path:
                print(p)
            return

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                heapq.heappush(pq, (heuristic(neighbor), neighbor, path + [neighbor]))

best_first_mc()
```

OUTPUT:—

```
gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/D
Solution Path:
(3, 3, 'L')
(2, 2, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(0, 2, 'L')
(0, 0, 'R')
gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```